Assignment 3 Course Project Documentation: Mahjong Game

Author: Amanda Li

Faculty of Arts and Science, University of Toronto

CSC209H1S: Software Tools and Systems Programming

April 7, 2026

Table of Contents

5.1 Project Overview

5.1.1 Application Description

The category that this application belongs to is Category 2 — Client–Server Application Using Sockets. It is a TCP client–server application that uses sockets and select to implement a simplified version of Mahjong, which is a 4-player tile-based turn-based tabletop game.

The server (server.c) keeps track of a linked list of clients (struct client on line 24) and a linked list/queue of ongoing games (typedef struct GameNode on line 34), and accepts clients and adds them to the next non-full game as a player. It accepts messages from clients about what move they choose to make, handles the game-logic and validates moves, and writes messages back to clients regarding information about the state of the game, including "hidden information" sent selectively to clients (hand tiles, tile drawn).

The client program (client.c) is run by a player. It handles user input made from the terminal (handle_user_input from line 462) and prints text to the terminal on the frontend , and communicates with the server and manages the client-side state of the game on the backend (handle_message from line 161).

5.1.2 Game Description

The game involves 136 tiles, where each tile has a category and a value, each represented by a character code. The categories are Dots ('D'), Bamboo ('B'), Characters ('C'), Winds ('W'), and Dragons ('G'). In terms of values, for Dots/Bamboo/Characters tiles, the possible values are the numbers from '1' to '9'. For Winds tiles, the possible values are 'n', 's', 'e', 'w'. For Dragons tiles, the possible values are 'r', 'g', and 'b'. For each possible combination of category and value, there are 4 tiles.

At the start of a game, the tiles are generated and shuffled, and 13 tiles are dealt to each player as their starting hand as hidden information known only to that player. The game begins with a random player, and ends when a player has a total of 14 tiles with at least 3 sets of three-of-a-kind tiles.

During the game loop, players take turns drawing and discarding ("playing") tiles. On their turn, players first draw a tile, then discard ("play") a tile. After a tile is discarded, other players can "steal" that tile to form a set of three-of-a-kind. The formed set of three tiles is then revealed to all players for the rest of the game, and can no longer be used by the player. If multiple players choose to steal a tile, the player who gets to steal the tile is the next player in turn order after the one who just played it.

5.1.3 Features and Interaction

1. **Join Game**: Run the client program to connect to the server and join the lobby of a game. Your player number and the number of players in the lobby will be received from the server and then output to the terminal.

```
wolf:~/Documents/group_53746/a3$ ./client teach.cs.toronto.edu
--------------------------------------------------------
Welcome, player 3.
Joining game, 3/4
```

2. **Start Game**: Once 4 players have joined that game, the game begins. The number of the player who starts, your own initial hand, and information about the game will be output.

```
The game has started. Player 4 starts. Enter Q to quit game, enter V to view current game information, enter H to view hand.
Your hand is:

 _  _  _  _  _  _  _  _  _  _  _  _  _  _
|W||B||W||W||B||D||D||B||C||B||C||D||W|
|s||4||w||n||2||3||3||1||9||1||4||4||e|
 -  -  -  -  -  -  -  -  -  -  -  -  -  -
```

Each Mahjong tile is formatted in a "rectangle" with the uppercase letter representing the category on the top and the character representing the value on the bottom. The following is what one Mahjong tile with category Wind ('W') and value 's' is printed as:

```
 _
|W|
|s|
 -
```

3. **Draw Tile**: The application will tell you when it is your turn to draw a tile. Enter d to draw a tile. After the server handles the action, the application will tell you which tile you drew.

```
Your turn to draw a tile. Type 'd' to draw a tile.
d
you wrote: d
------------------------------------------------------
You go to draw a tile
------------------------------------------------------
You drew B6.
```

4. **Play Tile**: The application will tell you when it is your turn to play a tile. Enter the tile you want to play in the format [category][value] where [category] is the character representing the category of the tile, and [value] is the character representing the value.

```
Your turn to play a tile. Enter the code of the tile you want to play.
C1
you wrote: C1
------------------------------------------------------
You go to play tile C1
------------------------------------------------------
You played C1.
------------------------------------------------------
You have played tile C1. Wait for other players to respond to your move.
```

5. **Respond to Play**: After another player plays a tile, you must make a response. Enter ‘p’ to pass, or steal by entering ‘s’ followed by the 2 tiles you want to reveal with the tile. These two tiles must be identical to the tile you want to steal. E.g. to steal tile C1, enter “sC1C1”

```
Player 1 played C1. Enter s to try to steal and p to pass. If you steal, also enter the two tiles to reveal with the stolen tile.
sC1C1
you wrote: sC1C1
------------------------------------------------------
You choose to steal, using tiles C1 and C1
------------------------------------------------------
You stole C1. Using C1 and C1
```

6. **View Hand and View Game Info**: At any time, you can view information about the current state of the game. Enter ‘H’ to view the tiles in your hand, and enter ‘V’ to view all information about the current game including your hand, your revealed tiles, and other players’ revealed tiles.

```
you wrote: V
------------------------------------------------------
Details of the current game: Player 2's turn. Enter Q to quit game, enter V to view current game information, enter H to view hand.
Your hand is:
 _  _  _  _  _  _  _  _  _  _  _  _  _
|W||C||C||B||D||D||C||D||B||D||C|
|s||9||8||2||2||1||8||6||8||9||3|
 _  _  _  _  _  _  _  _  _  _  _  _  _
Your revealed tiles are:
 _  _  _
|C||C||C|
|1||1||1|
 _  _  _
Player 1's revealed tiles are:



Player 3's revealed tiles are:



Player 4's revealed tiles are:



The discarded tiles are:
 _
|B|
|6|
 _
```

7. **Quit Game**: At any time you can enter ‘Q’ to quit the game. This ends the game and disconnects all players in that game.

```
you wrote: Q
------------------------------------------------------
You chose to quit game
------------------------------------------------------
Game has closed. Thanks for playing!
```

8. **Win Game**: When the tiles in your hand and your revealed tiles meet the win conditions, the game ends and every player is told who the winner is. However, the game does not close until a player enters ‘Q’ to quit the game.

```
You go to draw a tile
------------------------------------------------------
You drew C7.
------------------------------------------------------
Game is over, player 2 won! Enter 'Q' to leave.
```

5.2 Build Instructions

5.2.1 Build

Compile by running **make** with no arguments to use default port 4242, or run **make PORT=[port_number]** replacing [port_number] with the port number you want to use.

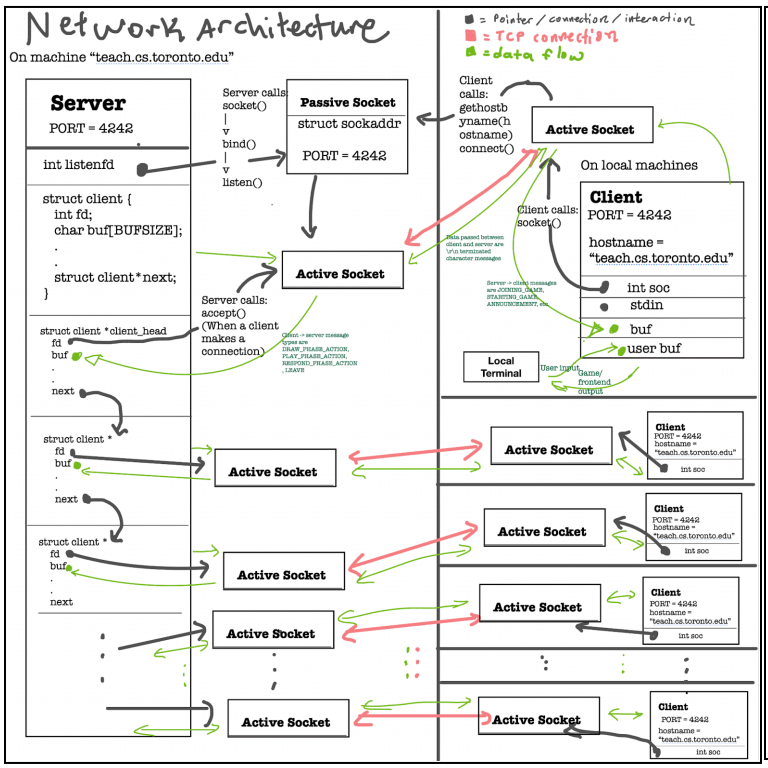
- Example command: **make PORT=58558**

5.2.2 Run

The server must be run before the clients. Run the server by running **server** with no arguments. For each client, run the client by running **client [hostname]** replacing [hostname] with the name of the machine on which the server is running. For example, if the server is running on a teach.cs.toronto.edu machine, run the client using **./client teach.cs.toronto.edu**

- Example command (server): **./server**
- Example command (client): **./client teach.cs.toronto.edu**

5.3 Architecture Diagram



The network architecture diagram on the left shows the full connections and interactions between the server and one client. The server initially sets up a listening socket, and the client sets up an active socket. Once a client tries to make a connection with the server using its hostname and port number, the server creates a new socket that has a TCP connection with the client's socket. Messages then travel both ways in the form of bytes treated as characters once received. In general, the server uses a linked list of client nodes to keep track of its connection to each individual connected client.

5.4 Communication Protocol

5.4.1 Code Implementation

In order to ensure clean, consistent communication between the server and clients, the project has a protocol.h and protocol.c file, which include structs and helper functions that allow for easy implementation of the communication protocol.

For every message, a certain process is followed, using the code from protocol.c/protocol.h:

1. The sender creates a payload containing the specific data to be sent (see section 5.4.3), then parses it into a character array using a specific parse helper function (e.g. typedef struct JoinPayload, parseJoinPayload(JoinPayload *payload_struct, char *payload, int direction=0))
2. The sender populates a Msg struct (see section 5.4.2), setting type to be the char representing the specific message type, and payload set to the newly created character array (e.g. Msg->type =JOINING_GAME, Msg->payload set to the previously formatted payload)
3. The sender encodes the Msg into a network-newline (\r\n) terminated character array using the encode function, and writes to the socket of the receiver (e.g. encode(Msg *msg, char *buf, int buffer_size), write(fd, buf, J_MSG_SIZE))
4. The receiver reads the message, using a buffer to get the entire network-newline (\r\n) terminated message, and decodes it into a Msg struct using the decode function (e.g. decode(Msg *msg, char *buf))
5. Based on the type, the receiver calls a parse helper function to parse the message payload character array into a specific payload struct (e.g. e.g. since msg->type == JOINING_GAME, parseJoinPayload(JoinPayload *payload_struct, char *payload, int direction=1))

* see lines 10, 17, 41-44, 46-49, 85, 86, 88-89 of protocol.h for the code mentioned above

5.4.2 Parent Message Structure: Msg

All messages between clients and server follow the same high-level message structure, and is represented by the Msg struct in the protocol.h file. Each message sent should be a network-newline (\r\n) terminated character array, and each receiver uses a buffer to find and process each message individually.

This communication structure was used because sending and interpreting all data as characters with each message terminated by a network-newline ensures clarity and consistency and reduces the potential risk of errors/mistakes. Messages of varying sizes can be easily sent and received, and having an encode and decode function ensures all messages are sent correctly.

Field	Description
Direction	Both ways (server to client and client to server messages both follow the same overall structure)
Encoding	A fixed-width binary struct encoded into a network-newline terminated character array (width depends on message type), written with a single write call: **typedef struct {** ** char type;** ** char payload[MAX_PAYLOAD];** // #define MAX_PAYLOAD 256 **}Msg;** Encoded form: **[type][payload]\r\n** (where [type] is a single character representing the message type, [payload] is a character array representing the payload data, and \r\n is the network-newline)

Semantics	Type tells the receiver what type of message is being sent. Payload is the content being sent, which is handled differently based on the type of message (see section 5.4.3).
Error Handling	If write returns −1, the sender logs the error and handles it based on the situation. - If the sender is the server, marks the client as disconnected (sets the player socket number to -1) and deletes the client, and ends the game that client was in, informing all clients in that game and deleting them as well. - If the sender is the client, it informs the player that the connection to the server was lost and ends the program. If write returns fewer bytes than the size of the Msg (1 + size of payload + 2), then that means a partial write occurred and only a part of the message was successfully sent. The program does not treat this as an error, and continues attempting to write the remainder of the message until the entire message is sent, or an error occurs (write returns -1). It is the responsibility of the sender to send correctly formatted, valid messages. If the receiver receives a message that overflows the buffer (buffer full but no network-newline found, which should not happen according to the communication protocol), then - If the receiver is a client, it will log the error and end the program. - If the receiver is a server, it will log the error, delete the client, and end the game that client was in, informing all clients in that game and deleting them as well. The server keeps running as normal.

5.4.3 Specific Messages: Message Types and Payloads

Each specific type of message has a constant char value representing the message type, a constant number representing the size of a message of that type (except for announcement type messages), and an encoding structure for the data written in its payload.

5.4.3.1 Server->Client Messages

Field	JOINING_GAME 'J'	STARTING_GAME 'S'
Encoding	#define JOINING_GAME 'J' #define J_MSG_SIZE 5 **typedef struct {** **int player_number;** **int players_found;** **} JoinPayload;** Payload encoding: **[player_number + '0'][players_found + '0']**	#define STARTING_GAME 'S' #define S_MSG_SIZE 30 #define STANDARD_HAND_SIZE 13 **typedef struct {** **int player_turn;** **Tile hand[STANDARD_HAND_SIZE];** **} StartPayload;** Payload encoding: **[player_turn + '0'][hand (represented as a char array, where each Tile is represented by 2 characters)]**
Semantics	Tells the client that the player is in the lobby of a game. The client tells the player that they are player **player_number** and that there are **players_found** players in the lobby.	Tells the client that the player's game has started. Client updates the client-side ClientGame struct, setting the curr_player (index) to player_turn-1 (player number), and populating the player's hand. The client program informs the player that the game has started and about the initial state of the game.

Field	ENDING_GAME ‘E’	DRAW_PHASE/ PLAY_PHASE ‘D’/’P’
Encoding	#define ENDING_GAME 'E' #define E_MSG_SIZE 4 **typedef struct {** **int winner; // 0 if no winner** **} EndPayload;** Payload encoding: **[winner + ‘0’]**	#define DRAW_PHASE 'D' #define PLAY_PHASE 'P' #define D_MSG_SIZE 4 #define P_MSG_SIZE 4 **typedef struct {** **int player_turn;** **} DrawPlayPayload;** Payload encoding: **[player_turn + ‘0’]**
Semantics	Tells the client that the game has ended. If winner is 0, client tells user that game has ended and ends the program. If winner > 0, client tells the client who the winner is.	Tells the client that it is now the draw/play phase. Player_turn is the player number whose turn it is. If it is the client’s player’s turn, prompts them to make a corresponding move.

Field	RESPOND_PHASE ‘R’	ANNOUNCEMENT ‘A’
Encoding	#define RESPOND_PHASE 'R' #define R_MSG_SIZE 6 **typedef struct {** **int last_player_turn;** **Tile tile;** **} RespondPayload;** Payload encoding: **[last_player_turn + ‘0’][tile (represented as 2 chars)]**	#define ANNOUNCEMENT 'A' #define DREW_TILE '0', #define PLAYED_TILE '1', #define STOLE_TILE '2', #define PASSED '3', #define ERROR '4' **typedef struct {** **int player_number;** **char action;** **int details_len;** **char details[MAX_PAYLOAD-4];** **} AnnouncementPayload;** Payload encoding: **[player_number + ‘0’][action][details_len][details]**
Semantics	Tells the client that it is now the response phase. Last_player_turn is the number of the player who just played a tile, and tile is the tile that was played. If last_player_turn does not represent the client’s player, it prompts the player to make a response. Otherwise, it tells the player to wait for other players’ response.	Represents an announcement about something that occurred in the game, that doesn’t need to prompt a new action from a player. Player_number is the player who did something, action is a code that corresponds to an announcement type (e.g. p1 drew a tile), details_len is the size of the additional details string, and details contains further data based on the type of announcement. **Data stored in details:** DREW_TILE: tile that was drawn sent to the client that drew it, otherwise nothing (send hidden information selectively) PLAYED_TILE: tile that was played STOLE_TILE: the 2 tiles used to steal a tile PASSED: nothing ERROR: an error message about a previous move made

5.4.3.2 Client->Server Messages

Field	DRAW_PHASE_ACTION ‘d’	PLAY_PHASE_ACTION ‘p’
Encoding	#define DRAW_PHASE_ACTION 'd' No additional data to send (this message only consists of the character code representing the message type and “\r\n”. I.e. a full draw message from a client is just “d\r\n”)	#define PLAY_PHASE_ACTION 'p' #define PLAY_MSG_SIZE 5 **typedef struct {** **Tile tile;** **} ClientPlayPayload;** Payload encoding: **[tile (2 chars)]**
Semantics	Tells the server that a player has chosen to draw a tile. The server checks to make sure that this is a valid move, then gives the player the next tile in the deck and sends an ANNOUNCEMENT and DRAW_PHASE message to the client. Ends the game if the player won or the deck is out of tiles, sends an error announcement if move is invalid.	Tells the server that a player has chosen to play a tile. The tile value represents the tile that the player chose to play. The server processes and validates the move and sends an ANNOUNCEMENT and RESPOND_PHASE message.

Field	RESPOND_PHASE_ACTION ‘r’	LEAVE ‘l’
Encoding	#define RESPOND_PHASE_ACTION 'r' #define RESPOND_MSG_SIZE_PASS 4 #define RESPOND_MSG_SIZE_STEAL 8 **typedef struct {** **char response;** // 'p' to pass, 's' to steal **Tile tiles[2];** // populated if steal **} ClientRespondPayload;** Payload encoding: **[‘p’]** or **[‘s’][tiles]**	#define LEAVE 'l' Leave has no additional data to send (this message only consists of the character code representing the message type and “\r\n”. I.e. a full leave message from a client is just “l\r\n”)
Semantics	Tells the server that a player has made a response. The server validates and processes the response, then sends an ANNOUNCEMENT and either DRAW_PHASE, PLAY_PHASE, or ENDING_GAME message based on the result.	Tells the server that a player has quit/left the game. Sends an ENDING_GAME message, Deletes that client, and the 3 other clients in the corresponding game, and then deletes the game.

5.5 Concurrency Model

5.5.1 server.c

The server achieves concurrency and avoids blocking on a single client by using a select loop handling the file descriptors of all connected clients as well as the file descriptor of the server’s listening socket. Since each client represents a player, the server is able to accept and process messages from all players concurrently, both for players within the same game and across games.

The select() loop in the main function of the server.c file works as follows:

1. Before entering the loop, set up the necessary variables like fd_set allset and fd_set rset. Clear allset with FD_ZERO then add the fd of the listening socket to allset using FD_SET. Set the max_fd (maximum fd in the set) to the fd of the listening socket. (see lines 90-91, 97-99)

2. At the start of the loop, reset rset to allset, so rset now holds the fds of all connected clients as well as the listening fd. This is necessary because select modifies rset each iteration. (line 103)
3. Call select(), passing in max_fd as nfds and rset as readfds. This monitors the file descriptors in rset and modifies rset to mark any fd that is ready for reading. (line 110)
4. Check if listenfd is ready for reading using FD_ISSET(listenfd, &rset). If it is ready, then a new client is attempting to connect. Handle this connection, and add the fd of the socket connecting to that client to allset. Update maxfd if necessary. (lines 127-239)
5. Then for each remaining ready file descriptor, parse the linked list of client nodes to find a client node corresponding to that file descriptor, and if found, that means a read is available from that client. Now, calling read on that fd will not block on a single client, and the result from the read can be handled accordingly. Update maxfd accordingly. (lines 241-299)
6. Repeat from step 2 until the program stops running.

5.5.2 client.c

Each client also uses select to achieve concurrency between handling messages from the server and user input from the terminal/standard input. It uses select on a set containing the fd of the socket that communicates with the server, and on STDIN_FILENO. This way it can monitor messages from the server and user without blocking. (see lines 62-74, 86-89, 91, 135)

5.5.3 Handling Partial or Multiple-Message Reads

A possibility with sending and receiving messages over a network is that a read message may be incomplete, or one read may contain multiple separate messages. The program handles these cases using a buffer (one buffer for server messages in client.c, and one buffer for each client node in server.c). Each time a read call is made, the data read is appended to the buffer and a loop checks for full messages (\r\n terminated messages) in the buffer, handling each message and stopping when there are no more full messages remaining. Then, the remaining data is stored in the buffer until the next loop iteration where read() is called again once more data is available, maintaining concurrency while retaining all data that is received.

5.6 Error Handling and Robustness

5.6.1 Bad Behaviour 1: Client disconnects mid-message or mid-game

If a client disconnected mid-message, then the read call in the handleclient function (line 317 of server.c) would return -1 for a client disconnect (lines 356-358 of server.c). Then, the line in the main function, “int result = handleclient(client_p);” (line 248 of server.c) would give result the value of -1. Then, lines 253-294 of server.c handle this by deleting the client, the game they were assigned to, the game node with that game, and all the players in that game, first sending a message to each deleted client telling them that the game has been ended. These actions are done through the delete_game function (see lines 274 and 928-969 of server.c), remove_gamenode function (see lines 205 and 465-492 of server.c), deleting the corresponding file descriptors (lines 279-281 of server.c), and updating the maxfd (lines 283-293 of server.c).

If a client disconnects mid-game and the server tries to write to the disconnected client, then the result of the write call would be -1. Then, any helper function that calls write has a return type of int, and

will return -1, which is received and returned by the handle_msg function (line 537 in server.c) which is the function that calls any helper function that might call write. The handle_msg function is called by handleclient, and so when handle_msg returns a value < 0, handleclient processes and returns that value. Then, the client and their corresponding game is deleted using the same code block as above (lines 253-294 of server.c).

For a client, when they receive an ENDING_GAME message from the server with value winner=0, they will stop the program. This ensures that after each other player in the game is deleted, the client side is also aware of the situation and can handle it.

Doing this ensures that even when a client disconnects and their game is unable to be played, the server is able to continue running as usual and all other games as well as future games are unaffected.

5.6.2 Bad Behaviour 2: Client sends an illegal move

Move validation is done on the client side, ensuring that a user's input move is legal, correctly formatted, and that it is their turn to make that move. If a user inputs an illegal move, the client should handle it and ask the user to make a correct move. However, in the event that this is bypassed and the client sends an illegal move to the server, the server is able to handle it accordingly without crashing or ending a game, or allowing illegal moves to be made, and informs the client so that they can remake that move or know that they were unable to make that move

If a client sends a move that they should not be able to make, for example making a move when it is not their turn, playing a tile that is not in their hand, or stealing a tile using tiles that are not in their hand, then the server handles this in the handle_msg function. For example, in the case PLAY_PHASE_ACTION (lines 585-631 of server.c), the server checks to make sure that it is that player's turn (player->game->curr_player == player->player_number-1), and that the tile being played exists in that player's hand (if (tile_found)). If not, it makes an announcement with type ERROR and an error message (**make_announcement(player->game, ERROR, player->player_number, "not your turn to play card")** or **make_announcement(player->game, ERROR, player->player_number, "tile not found")**). The make_announcement function (lines 1078-1188 of server.c) then sends a message with type ANNOUNCEMENT and an error announcement payload with the error message to that client only.

When the client receives an error announcement, it resets its local pending_move variable back to the previous value so the user can remake the move if applicable, and prints the error message to the terminal.

5.6.3 Bad Behaviour 3: EOF or error receiving from standard input or server

It is also possible for bad behaviours to occur on the client side. The client catches issues with using read on the server (line 93 of client.c) or using fgets on stdin (line 138 of client.c), by checking the return values of these calls. Since these calls are made in the main loop, which uses a while loop that checks a flag running on each iteration, if the return value of read is <= 0, or the return value of fgets is NULL, the lines **running = false;** and **continue;** are run. This ends the loop, and after exiting the loop, the program performs cleanup by freeing any dynamically allocated memory (line 155 of client.c) and closing the socket connection to the server.

5.7 Team Contributions

Amanda Li: I completed this project individually, so I did every part of the project.